Algorithm Documentation

How it works · Line by line · From zero to expert

■ RSI Momentum	■ EMA Trend Filter	■ Stop Loss	■ Take Profit

Strategy	RSI + EMA Cross
Timeframe	5-minute candles
Coins watched	BTC, ETH, SOL, BNB, ADA
Stake per trade	100 USDT
Max open trades	3
Stop loss	-5% (max loss: 5 USDT per trade)
Max take profit	+4% (4 USDT per trade)

1 The Big Picture — What is the Strategy Doing?

At its core, this strategy answers one question every 5 minutes for each coin:

> *"Has this coin dropped fast enough to be a good buying opportunity, and is the overall trend still going upward?"*

If the answer is **yes** — it buys. Then it waits for the price to recover and sells at a profit, or cuts the loss quickly if the price keeps falling.

Why this makes sense

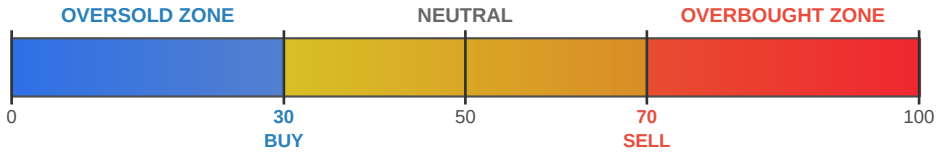
Markets don't go up in a straight line. Even in a healthy uptrend, prices regularly **dip** — and then recover. This strategy tries to catch those dips and ride the recovery.

Think of a coin's price like a hiker climbing a mountain. They don't walk straight up — they occasionally stop

2 Indicator 1 — The RSI (Relative Strength Index)

RSI is a number between **0 and 100**. It measures **how fast and how much** a price has moved recently. It does not tell you the direction — it tells you the **speed and intensity** of the move.

The RSI scale



RSI Range	Market condition	What the strategy does
0 – 30	OVERSOLD — price dropped too fast	■ Potential BUY signal
30 – 70	NEUTRAL — normal market movement	— Do nothing, wait
70 – 100	OVERBOUGHT — price rose too fast	■ Potential SELL signal

How RSI is calculated (the idea, not the math)

Over the last **14 candles** (14 × 5 minutes = 70 minutes), the formula asks: *"On the candles that went up, how much did they go up on average? And on the candles that went down, how much did they go down?"*

If the ups were much bigger than the downs → RSI is high (overbought).

If the downs were much bigger than the ups → RSI is low (oversold).

In the code

```
def populate_indicators(self, dataframe, metadata):
    # Calculate RSI using the last 14 candles
    dataframe["rsi"] = ta.RSI(dataframe, timeperiod=self.rsi_period.value)
    #                                                ^ this is 14 by default
    ...
```

✓ The RSI period of 14 is an industry standard introduced by J. Welles Wilder in 1978. Most traders use 14. Shorter periods (e.g. 7) react faster but give more false signals. Longer periods (e.g. 21) are slower but more reliable.

3 Indicator 2 — The EMA (Exponential Moving Average)

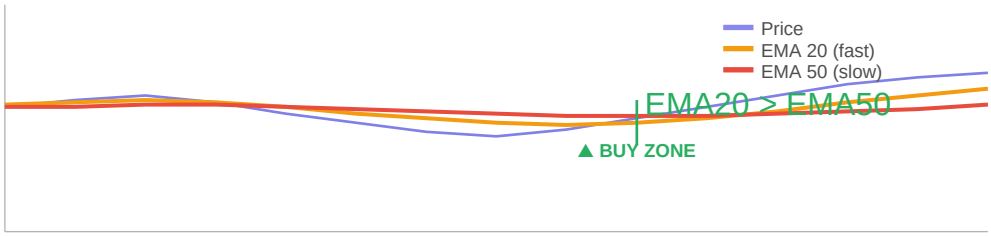
A Moving Average smooths out the noisy price data into a single line that shows the **general direction** (trend) of the price. The **Exponential** version gives more weight to recent prices, making it react faster to changes than a simple average.

EMA 20 vs EMA 50

	EMA 20 (Fast)	EMA 50 (Slow)
Looks back at	Last 20 candles (100 min)	Last 50 candles (250 min)
Reacts to price	Quickly	Slowly
Shows	Short-term trend	Long-term trend
Color in diagram	Orange	Red

The crossover signal

When the **fast EMA (20) is above the slow EMA (50)**, it means the recent price is higher than the longer-term average — the trend is **going up**. This is called a **Golden Cross** and is used as a trend confirmation filter.



EMA crossover — when the orange line rises above the red line, the trend is up

In the code

```
def populate_indicators(self, dataframe, metadata):
    ...
    dataframe["ema_20"] = ta.EMA(dataframe, timeperiod=20)  # fast line
    dataframe["ema_50"] = ta.EMA(dataframe, timeperiod=50)  # slow line
    return dataframe
```

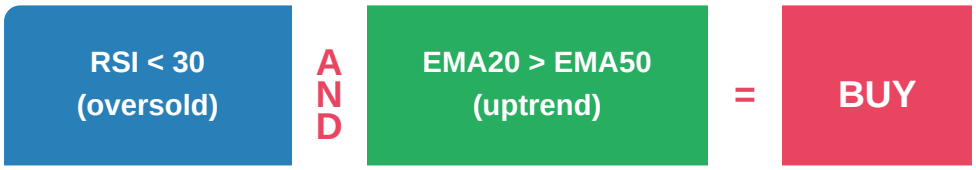
The EMA filter is what separates this strategy from a naive RSI strategy. Without it, the bot would buy every dip — including dips in a strong downtrend, which often keep falling. The EMA makes sure we only buy dips in uptrends.

4 Entry Signal — When the Bot Buys

This is the most important part of the strategy. The bot opens a trade **only when both conditions are true at the same time**:

Condition	Indicator	Threshold	Meaning
1 — RSI oversold	RSI (14)	RSI < 30	The coin has dropped fast — potential bounce
2 — Uptrend confirmed	EMA 20 / 50	EMA20 > EMA50	The general trend is still going up

The AND logic — both must be true



In the code

```
def populate_entry_trend(self, dataframe, metadata):
    dataframe.loc[
        (
            (dataframe["rsi"] < self.buy_rsi.value) &   # Condition 1: RSI < 30
            (dataframe["ema_20"] > dataframe["ema_50"]) & # Condition 2: uptrend
            (dataframe["volume"] > 0)                   # sanity: market is active
        ),
        "enter_long",
    ] = 1  # 1 = "yes, open a trade on this candle"
    return dataframe
```

What 'enter_long' means

Long means you expect the price to go **up**. You buy now and sell later at a higher price. This strategy only trades long — it never bets on prices going down (no shorting).

> The volume > 0 check ensures the bot doesn't trade on candles with zero activity (e.g. during exchange downtime). It's a basic sanity filter.

5 Exit Signal — When the Bot Sells

The strategy has **three independent ways** to close a trade. Whichever triggers first wins:

#	Exit method	Trigger	Type
1	RSI exit signal	RSI rises above 70 (overbought)	Sell signal
2	Minimal ROI — instant	Profit ≥ +4% at any time	Take profit
3	Minimal ROI — 30 min	Profit ≥ +2% after 30 minutes	Take profit
4	Minimal ROI — 60 min	Profit ≥ +1% after 60 minutes	Take profit
5	Minimal ROI — 2 hours	Profit ≥ 0% after 2 hours	Break-even exit
6	**Stop loss**	**Loss reaches -5%**	**Risk management**

The Minimal ROI logic explained

The bot becomes **progressively less greedy** the longer a trade stays open. This prevents trades from being held forever waiting for a big profit that never comes:

Time open	Minimum profit to exit	Reasoning
0 – 30 min	+4%	Fresh trade: only exit on a strong move
30 – 60 min	+2%	Getting older: accept smaller profit
60 – 120 min	+1%	Stale trade: take anything positive
120+ min	+0%	Very stale: exit at break-even to free capital

The stop loss

If the price drops **5% below the entry price**, the bot exits immediately with a **market order** — meaning it sells at whatever the current price is, instantly. This is the most important risk management tool.

```
stoploss = -0.05  # -5%

# Example: bought BTC at 100,000 USDT
# Stop loss triggers at: 100,000 × (1 - 0.05) = 95,000 USDT
# Max loss on this trade: 100 USDT × 5% = 5 USDT
```

In the code — exit signal

```
def populate_exit_trend(self, dataframe, metadata):
    dataframe.loc[
        (
```

```
            (dataframe["rsi"] > self.sell_rsi.value) &  # RSI > 70
            (dataframe["volume"] > 0)
        ),
        "exit_long",
    ] = 1  # 1 = "close the trade on this candle"
    return dataframe
```

6 The Complete Strategy — All Together

Here is the full flow of what happens every 5 minutes for each coin:

Step	Action	Detail
1	New 5-min candle closes	Binance sends OHLCV data to the bot
2	Calculate indicators	RSI(14), EMA(20), EMA(50) computed on latest data
3	Check entry conditions	RSI < 30 AND EMA20 > EMA50?
4a	YES → open trade	Buy 100 USDT of the coin at limit price
4b	NO → do nothing	Wait for next candle
5	Monitor open trades	Check ROI and stop-loss every candle
6	Exit condition met?	RSI > 70, ROI reached, or stop-loss triggered
7	Close trade	Sell at limit (or market for stop-loss)
8	Repeat	Go back to step 1

The complete strategy file

```
class RSIStrategy(IStrategy):

    timeframe = "5m"          # check every 5 minutes

    minimal_roi = {           # exit if profit reaches:
        "0":   0.04,          #   4% at any time
        "30":  0.02,          #   2% after 30 min
        "60":  0.01,          #   1% after 60 min
        "120": 0              #   0% after 2 hours
    }

    stoploss = -0.05          # exit if loss reaches -5%

    buy_rsi  = IntParameter(default=30, ...)  # buy  when RSI < 30
    sell_rsi = IntParameter(default=70, ...)  # sell when RSI > 70

    def populate_indicators(self, df, meta):
        df["rsi"]    = ta.RSI(df, timeperiod=14)
        df["ema_20"] = ta.EMA(df, timeperiod=20)
        df["ema_50"] = ta.EMA(df, timeperiod=50)
        return df

    def populate_entry_trend(self, df, meta):
        # BUY when RSI < 30 AND short EMA above long EMA
```

```
        df.loc[(df["rsi"] < 30) & (df["ema_20"] > df["ema_50"]),
               "enter_long"] = 1
        return df

    def populate_exit_trend(self, df, meta):
        # SELL when RSI > 70
        df.loc[df["rsi"] > 70, "exit_long"] = 1
        return df
```

7 Limitations & How to Improve

No strategy is perfect. Understanding the weaknesses helps you improve it:

Limitation	Problem	Possible fix
RSI can stay oversold	In a strong crash, RSI stays below 30 for days. The bot keeps buying a falling market.	Add a 200-period EMA filter: only trade if price is above EMA200
No volume confirmation	A dip on low volume is less reliable than one on high volume.	Add volume > volume.rolling(20).mean()
Fixed stop loss	5% stop loss ignores volatility. High-volatility coins need wider stops.	Use ATR-based dynamic stop loss
Only long trades	The bot makes no money in bear markets.	Add short-selling logic (requires futures/margin)
No news awareness	A coin crashing due to bad news will trigger RSI < 30 and the bot will buy.	Hard to fix automatically. Consider pausing bot during major events.

Tunable parameters

These are the knobs you can turn to change behaviour. Always backtest after changing any of them:

Parameter	Default	Lower value effect	Higher value effect
RSI period	14	Reacts faster, more signals, more noise	Reacts slower, fewer but stronger signals
Buy RSI	30	Buys only extreme dips (fewer trades)	Buys earlier dips (more trades, more risk)
Sell RSI	70	Sells at smaller recoveries (less profit)	Waits for bigger recoveries (more risk)
Stop loss	-5%	Cuts losses faster (safer, misses bounces)	Allows bigger drawdowns (riskier)
Timeframe	5m	More signals, more noise, more fees	Fewer signals, cleaner, lower fees

✓ Freqtrade has a built-in tool called **Hyperopt** that automatically tests thousands of parameter combinations on historical data to find the optimal values. Ask for help running it when you're ready.